Phase 3 specs: VP helpers, small items, M-GMM

Rev 1, 2026-10-21 (ET). Written for Coder. Checked against `github.com/ettiennehugo/MultiCharts.Net` `main` @ `e37a933` (PR #17 merged). MC API facts are taken from the Programming Guide (`/workspace/mc-docs/01M5BF386FDT0VH0AYQBMXHEF2.pdf`, "PG"). Anything the PG does not state is marked **PROBE**.

Sources: `trading-idea-catalogue.md` (§2.4a and the Needs lines), `specs/missing-functionality-specs.md` rev 4 ("MFS"), `specs/missing-functionality-review-analyst1.md`, `reviews/analyst-base-class-spec.md` rev 2.7 ("BCS"), `specs/m-port-spec.md`, and `trading-ideas-analyst2.md` ("A2").

Out of scope: M-GEX and M-NEWS, both on hold by the user. The A2-33 harness is being built by Coder. Its shared interfaces are covered in §0.6.

---

0. Cross-cutting rules (inherited, plus what is new)

0.1 Rules carried over unchanged

- **Laziness (MFS §0.1).** Each engine is an `internal sealed class` (or a static helper with no state). It is created in `StartCalc()` only when its protected config says it is enabled, and is `null` otherwise. **No new `[Input]`s.** A disabled engine's accessors return neutral values (`null`, `NaN`, `Unknown`) and log one Warning (the `m_paQueryWarned` pattern).
- **Closed bars only.** Commit on `Bars.Status == EBarState.Close`, de-duplicated per stream by `CurrentBar` (PG p20, p35).
- **Recalc idempotence per Data1 bar** (MFS §0.4, PG p54).
- **Journal.** Facts go to `ETraceLevel.Journal` through `engineSink(ETraceLevel.Journal)`. They are stamped with **bar times** (`yyyy-MM-dd HH:mm:ss`, chart TZ), never bar indexes. Per-bar detail is logged at `Debug`. A `null` sink means no string is formatted.
- **Rolls (MFS §0.3).** dxFeed `@ES` is continuous and unadjusted. A roll **invalidates**; it never shifts. No multi-session statistic may span a splice (A2 "Roll rule", A2-39).
- **Sessions.** 16:00 ET is the value end (`RthValueEndEt`, which handles early closes via `RthEarlyCloseEt`). 16:15 is the session-end flag. Holidays come from `IsGlobexClosedDay` / `IsNoRthDay`.
- **Costs** use MC native commission and slippage plus `CostFilterMaxFractionOfR` (no change).
- **MTF** uses native `BarsOfData(n)` with `HtfClosed` (MFS §3). No internal HTF bar building.
- **Code style.** The test projects compile with **`LangVersion 7.3`**, so use no C# 8+ features (no switch expressions, no `??=`, no nullable reference types, no ranges). New pure logic goes inside `// <tag>` … `// </tag>` regions so the `tests/*/run.sh` extract-and-compile pattern works. Use no LINQ in per-bar paths and no per-bar allocation in hot loops.

0.2 New shared piece: the closed-bar ring (`_BarRing`)

Many helpers below need closed OHLCV of a stream for arbitrary past windows. MaxBarsBack applies to `Bars.X[n]`, and its per-series behaviour is unverified (MFS §3.5, Probe P2). So the helpers do not index back through `Bars`. Instead they read a ring that the host fills once per closed bar.

- **File:** `_BarRing.Function.CS`, region `// <bar-ring>`.
- **Record:** `struct RingBar { int bar; DateTime time; DateTime tradingDay; double o, h, l, c, v, up, down; bool rth; bool rollBar; }`. up/down are `Bars.UpTicks/DownTicks`, used only for the BarUpDown tier.
- **API:** `Add(RingBar)`, `int Count`, `RingBar this[int ago]`, `int IndexAtOrBefore(DateTime t)` (binary search over a monotone `time`), `bool Covers(DateTime t0)`.
- **Capacity:** `BarRingConfig.capacityBars`. The default is **0 = auto**: the sum of every enabled consumer's need, capped at 40,000 (≈ 29 Globex sessions of 1m). The host keeps one ring per stream, created **only** if some

enabled helper asks for that stream.
- **Edge cases.**
- A window starting before the oldest ring bar gives a profile with `isPartial = true`. The caller decides whether to use it; the helpers never fabricate data.
- `rollBar` is set from `rollOn(ds)`. Data gaps are not filled. `coverage` metrics (below) report them.
- **Journal:** `RING Data{n} capacity({cap}) consumers({list})`, written once in StartCalc.

0.3 Roll detection moves into `_CustomSignal` (needed by S4 and S5)

See S4. All helpers below call `rollOn(ds)`. After S4, that works on any base.

0.4 Deterministic PRNG (`_Rng`)

The bootstrap, KDE restarts, GMM k-means++ and the AVWAP P2 placebo need random numbers that are reproducible across .NET runtimes. `System.Random`'s algorithm is a runtime implementation detail and must **not** be used.

- **File:** `_Rng.Function.CS`, region `// <rng>`. SplitMix64: `state += 0x9E3779B97F4A7C15; z = state; z = (z ^ (z >> 30)) * 0xBF58476D1CE4E5B9; z = (z ^ (z >> 27)) * 0x94D049BB133111EB; return z ^ (z >> 31)`.
- **API:**
- `NextDouble()` = `(next() >> 11) * 2^-53`.
- `NextInt(n)` uses Lemire rejection.
- `static ulong Seed(params long[] parts)` hashes its parts through SplitMix64.
- **Seed convention:** every use seeds from `(componentId, tradingDay.Ticks, configHash)`. Results therefore depend only on the data, never on the run.
- **Unit test:** a seed-0 vector test (Coder pastes the vectors from the reference implementation; the Python port in G-V must reproduce them).

0.5 Config hash

Each new config class implements `string Canon()`, which returns its fields in fixed order. The journal's per-run config hash (catalogue §2.5) includes them. This means the `Variant` sweeps are fully identified.

0.6 Interfaces shared with Coder's A2-33 harness

The A2-33 harness scores levels. Every helper here that emits a level emits the same record, so the harness can consume levels without adapters:

```
public enum ELevelKind { Poc, Vah, Val, Hvn, Lvn, RangePoc, RangeVah, RangeVal, WeekPoc, WeekVah,
WeekVal, FvgUpper, FvgLower,
                         GmmMu, GmmEdge, GmmVoid, SigmaMu, VtPrice, BootstrapPocZoneLo,
BootstrapPocZoneHi }
public sealed class LevelRef {
    public ELevelKind kind; public double price, lo, hi;   // lo/hi = zone, both = price for a line
level
    public DateTime createdTime;        // bar time from which it is usable (no lookahead)
    public DateTime sourceT0, sourceT1; // window that produced it
    public int dataStream; public string tag;  // e.g. "RP:20261021T0930-1130", "WK:2026W42",
"FVG:U:202610211005"
    public double quality;              // helper-specific (stability, π_k, gap ticks); NaN if none
    public bool crossesRoll;
}
```

- **Type and file:** put this in `_LevelRef.Function.CS`, region `// <level-ref>`. If the harness already defines an equivalent type, **adopt it and rename these fields instead**. That is open question Q1.
- **Lookups:** the harness also needs matched-control prices and level ages. It should read `RangeProfile`, `WeeklyProfiles`, `_FvgDetector` and `_ProfileStats` through the APIs below, not through private state.

1. VP helpers (priority 1)

VP-1 Range / composite profile over arbitrary bar ranges

Purpose. A volume profile over any closed-bar window [t0, t1] on a stream. Examples: a breakout leg, a balance zone, the last 30 minutes, an AVWAP anchor to now. This window does not have to match a session or `Composite(n)` of whole sessions.

Used by: - PS-13: accumulation zone [t0, t1], closed bars only. - PS-12 variant: range profile "as DV-03". - PS-14: range profile. - A2-15: breakout-leg POC from the leg AVWAP anchor. - A2-30: rolling 30-min profile, through VP-4. - A2-29 / A2-31: developing RTH 09:30–t profile, used as F-DEV on Own-dev. - A2-35: arbitrary window. It is gated by M-NEWS (deferred), but the helper is generic. - VP-4 (entropy, KL), VP-6 (bootstrap / KDE / V/T) and the GMM profile mixtures (G-P) all build their histograms from VP-1.

Classes and files. - `_RangeProfile.Function.CS`, region `// <range-profile>`. It holds `RangeProfileConfig`, `RangeProfileBuilder` (pure), and `RangeProfileHandle` (incremental and open-ended). - Host API on `_VolumeProfileBase` (it reuses `fillFromHistogram` so POC/VA/HVN/LVN match `Composite`).

Config (protected property `RangeProfileConfig RangeProfiles`, default `enabled = false`).

Field	Default	Meaning
`enabled`	false	Creates the Data n rings in `streams`.
`streams`	{1}	Streams that may be profiled.
`distribution`	2 (uniform)	Same codes as `ProfileMath.Accumulator.Add(mode)`: 1 = close, 2 = uniform over [L, H] at 1-tick rows, 3 = typical. Uniform is A2's convention.
`useTickLadder`	true	If a VP-3 ladder exists for a bar (tier ≤ TickRule), use its exact per-row volumes instead of the bar distribution. The result's `approximate` flag is false only if **all** bars used a ladder.
`rthOnly`	false	Include only `isRthBar` bars.
`rollPolicy`	`Reject`	`Reject` returns null if a roll bar is inside (t0, t1]. `Truncate` starts at the roll bar and sets `isPartial`.
`valueAreaPercent`	base `ValueAreaPercent`	Same VA method as the base.
`maxRows`	4000	Guards against a bad tick size.
`cacheSize`	8	LRU of built profiles keyed by (stream, t0, t1, lastBar).
`maxOpenHandles`	8	Open anchored handles.

API (on `_VolumeProfileBase`).

```
public VpExtra RangeProfile(int stream, DateTime t0, DateTime t1);         // closed bars with t0 <=
time <= t1 (time = MC bar stamp)
public VpExtra RangeProfileBars(int stream, int barsBack, int count);      // by ring offsets
public RangeProfileHandle OpenRange(int stream, DateTime t0, string tag);  // anchored, grows with
each closed bar
public RangeStats RangeStatsOf(VpExtra e);  // {bars, expectedBars, coverage, ladderBars, rows,
totalVol}
```

- The return is a `VpExtra` with `isOwn = true`, `approximate`, `isPartial` and `crossesRoll`, plus POC/VAH/VAL/HVN/LVN through `fillFromHistogram`, exactly as in `Composite` (`_VolumeProfileBase` l.1221).
- `RangeProfileHandle`: `Step()` is called by the host on each closed bar. It exposes `Profile` (a VpExtra rebuilt lazily, only when it is read and is dirty), `Close(DateTime t1)` (freezes it) and `Tag`. Closed handles are dropped after 5 sessions.

Algorithm. 1. Resolve the indexes `i1 = ring.IndexAtOrBefore(t1)` and `i0 = first index with time ≥ t0`. If `i0 > i1`, return null. 2. If any bar in (i0, i1] has `rollBar`, apply `rollPolicy`. 3. Build a `ProfileMath.Accumulator(tick)`. Each bar either goes through `Add(..., distribution)` or, if a ladder exists and `useTickLadder` is set, through one row add per ladder row (total). **Change to `_ProfileMath`:** make the private `addRow` public as `AddRow(long key, double v)`. 4. Call `Dense()`, then `fillFromHistogram(e, dp, dv)`. Set `e.startBar` / `e.endBar` from the ring bar numbers, `open`/`high`/`low`/`close` from the bars, and `levels` from the dense rows (`total`; `ask`/`bid` only if every bar had a ladder, else NaN). 5. Coverage: `expectedBars` = the count of stream-resolution slots in [t0, t1] where the session clock is open (`GetGlobexState != Closed`; skips the daily halt, holidays and early closes). Then `coverage = bars / expectedBars`. When coverage < 0.95, journal once per handle.

Laziness. No ring and no work happens unless `RangeProfiles.enabled` is set or a dependent (VP-4, VP-6, G-P) forces it on. A profile is built only when it is requested. The cache makes a repeat query on the same calc bar O(1). A handle step is O(rows of the new bar).

Edge cases. - **Roll:** see `rollPolicy`. A handle that is open at a roll is closed with `crossesRoll = true` and journaled. - **Holidays / half days:** the window spans real bars only, and `expectedBars` honours the early close (13:00 does not show as a gap). - **Data gaps:** reported through coverage and never interpolated. - **t1 in the future, or the forming bar:** never included. - **t0 older than the ring:** `isPartial = true`, plus one journal line.

Journal. `RP build {tag} Data{n} [{t0}..{t1}] bars({b}/{exp}) poc({p}) va({val}-{vah}) approx({0|1}) roll({0|1})`. This is written once per handle close. Ad-hoc calls log only at `Debug`.

Unit tests (`tests/RangeProfileTests`). 1. Tick 0.25: - bar A: H 100.50, L 100.00, V 300, giving 100 per row on 3 rows; - bar B: H = L = 100.25, V 50; - bar C: H 101.00, L 100.75, V 20.

Expect POC = 100.25 (150), total = 370 and 5 rows (100.00 … 101.00). 2. Close mode vs uniform mode on the same bars. 3. `Reject` returns null when a roll bar is inside the window. `Truncate` starts at the roll bar. 4. Coverage: 390 RTH slots with 5 bars removed gives 385/390. 5. Handle incremental equals a one-shot build exactly after 500 seeded random bars. 6. A ladder bar overrides the uniform distribution.

MC acceptance trace (A-RP). ES 1m, `RangeProfile(1, 09:30, 16:00)` vs the MC RTH session VP on 20 sessions, logging POC/VAH/VAL for both. - On minute data the target is POC within ±1 tick on ≥ 90% of sessions. This figure is report-only because MC's own minute distribution is unknown (BCS VPB-1). - Inside the tick window with `useTickLadder`, the target is ≥ 99%, the VPB-1 criterion.

Dependencies: `_BarRing`, `_ProfileMath` (AddRow), `fillFromHistogram`, `rollOn`, optional VP-3.

VP-2 Weekly profile stream

Purpose. Completed and developing **weekly** profiles plus N-week composites. This is the "VPF-3 extension" in the catalogue.

Used by: - MT-02: weekly accepted value plus a 60m trigger. - SW-01: weekly balance break on a 3–6-week composite. - DV-10 can also use it for weekly context. The weekly VWAP already exists as `EVwapType.Week`.

Classes and files. `_WeeklyProfiles.Function.CS`, region `// <weekly-profiles>`, containing `WeeklyProfileConfig` and `_WeeklyProfiles`. The host is `_VolumeProfileBase`.

Config (`protected WeeklyProfileConfig WeeklyProfiles`).

Field	Default	Meaning
`enabled`	false	
`dataStream`	1	
`source`	`RthSessions`	`RthSessions` merges each session's RTH (value-window) profile. `GlobexSessions` merges full Globex days.

Field	Default	Meaning
`weekKey`	Monday-start week of `tradingDayKey`	The same key `_AvwapManager` uses for Week resets, so weekly VWAP and weekly VP agree.
`keepWeeks`	8	Completed weeks kept.
`minSessions`	3	A week with fewer sessions is published with `isPartial = true`.
`includeDeveloping`	false	Whether `Week(0)` includes the developing session.
`rollPolicy`	`Reset`	A2-39 default. The cash-terms and back-adjusted arms are not built (Q3).

API.

```
public VpExtra Week(int weeksAgo);            // 0 = developing week
public VpExtra WeeklyComposite(int nWeeks);   // last nWeeks COMPLETED weeks; null if fewer than
nWeeks after the last roll
public int CompletedWeeks { get; }
public DateTime WeekKey(int weeksAgo);
```

Algorithm. - On the first closed bar of a new `tradingDayKey`, take the just-completed session profile. That is the `VpExtra` from `RecentCompletedProfiles(stream, 1, cur)`, using the RTH/Own-dev profile when `source = RthSessions`. - Fold its `levels` into the week's `SortedDictionary<long, double[]>` (row → {total, ask, bid}). - A new week key completes the prior week: build a dense grid, run `fillFromHistogram`, and push it onto a ring of `keepWeeks`. - `WeeklyComposite(n)` merges the stored week histograms row-wise, cached per (n, newest week). - The cost is O(rows) per session. Nothing happens per bar.

Laziness. It is created only when enabled, and works once per session close.

Edge cases. - **Roll (Reset):** the week containing the roll is split. The pre-roll part gets `crossesRoll` and is not usable. The post-roll part starts a new week with `isPartial = true`. The composite never spans a roll. One journal line is written. - **Holiday weeks** (Thanksgiving, Christmas, New Year): handled by `minSessions`. - **Half days:** included, because they contribute real volume. - **Sunday evening:** belongs to Monday's Globex date. - **Year boundary:** the key is the Monday-start date, not the ISO week number. Test 2026-12-28 → 2027-01-01 as one week. - **A partial session** inside a week makes the week `isPartial`.

Journal. `WK complete {mondayDate} sessions({k}) poc({p}) va({val}-{vah}) roll({0|1}) partial({0|1})`.

Unit tests. 1. 5 synthetic sessions give a weekly sum equal to the row sum. 2. A holiday week with 2 sessions is partial. 3. The year-boundary key. 4. A mid-week roll splits the week. 5. A 3-week composite equals the row-wise sum, and is null across a roll.

MC trace (A-WK). Log the weekly POC/VAH/VAL for 12 weeks next to an MC Volume Profile study set to Weekly. The comparison is report-only, because MC's weekly session boundary may differ.

Dependencies: the session-profile list in `_VolumeProfileBase`, `tradingDayKey`, `rollOn` and `fillFromHistogram`.

VP-3 Per-bar profile ladder (bid/ask volume per price per bar)

Purpose. Every closed bar gets a price ladder: rows with bid volume, ask volume and total. The ladder is tagged by the rev 2.7 order-flow quality chain and can be aggregated to HTF bars (the 30m ladder PS-12 asks for).

Used by: - PS-12: HVN on per-bar 30m profiles ("✗ per-bar profile access"). - OF-04: absorption at tier ≥ 3. - OF-06: unfinished auction at tier ≥ 3. - A2-38: λ̂ at tier ≥ 3. - VP-1's `useTickLadder` and VP-6 V/T in W-T.

Classes and files. `_BarLadder.Function.CS`, region `// <bar-ladder>`, containing `LadderBar`, `BarLadderConfig` and `_BarLadderStore`. It sits on top of `_OrderFlow.FootprintBar`.

Rev 2.7 quality chain (BCS rev 2.7 §B; S1 adds tier 2).

Rank	`FlowSource`	Ladder	Quality
1	Aggressor	reserved, never detected (not exposed by MC .NET, BCS §A3)	Exact
2	**McBidAsk** (S1)	MC Ask/Bid-traded levels	High
3	Quote	our Lee-Ready	High
4	TickRule	1-tick trades	Medium (Low at bar extremes; `excludeExtremes`)
5	BarUpDown	totals only (uniform over [L, H]); bid/ask = NaN	Low
6	Unavailable	none	n/a

Record.

```
public sealed class LadderBar {
    public int bar; public DateTime time; public double tick;
    public _OrderFlow.FlowSource source; public _OrderFlow.FlowQuality quality;
    public long[] rows; public double[] bid, ask, total;   // ascending rows, parallel arrays
    public double High, Low, Buy, Sell, Volume, coverage;  // Buy = Σask, Sell = Σbid; coverage for
HTF bars
    public bool crossesRoll;
    public bool HasBidAsk { get { return source <= _OrderFlow.FlowSource.TickRule; } }   // relies
on the S1 enum order
}
```

Config (`protected BarLadderConfig BarLadders`).

Field	Default	Meaning
`enabled`	false	Forces `OrderFlowEnabled = true` on the futures base. On a non-futures base, only BarUpDown is available, and a one-time Warning is logged.
`keepBars`	2000	Data1 bars kept.
`htfStreams`	{}	Data n streams that get an aggregated ladder.
`minQuality`	Low	Consumers may pass a stricter value.

API.

```
public LadderBar Ladder(int ago);                          // Data1, from FlowEngine.Closed(ago) (+ S1
level set)
public LadderBar LadderAt(DateTime barTime);
public LadderBar HtfLadder(int htfStream, int ago);      // aggregate of Data1 ladders in
(prevHtfTime, htfTime]
public VpExtra LadderProfile(LadderBar b);               // the bar's rows through
fillFromHistogram, so HVN/LVN nodes match the session logic (PS-12)
```

Algorithm. - **Data1:** on the first query for a closed bar, convert `FootprintBar.Ladder` (`row → {bid, ask}`) to sorted arrays and memoise them. When S1 has a matching MC level set, McBidAsk is used. - **BarUpDown:** total = V spread uniformly over the rows of [L, H]. Bid and ask are NaN. - **HTF:** - For Data n bar k (closed, `HtfClosed`), the window is (Time[k+1], Time[k]], which assumes MC close stamps. **That needs Probe P2.** - Merge the Data1 ladders in the window row-wise. - `source` = the worst (max) source of the parts, and `quality` = the worst quality, per BCS: a cumulative value carries the minimum quality. - `coverage` = Data1 ladders found / expected Data1 slots. If coverage < 1, bid/ask are set to NaN and the source becomes BarUpDown (totals are kept from the ring bars).

Laziness. It is not built unless enabled, and the conversion is memoised per queried bar.

Edge cases. - **Roll:** nothing resets. An HTF bar that contains the splice is flagged `crossesRoll`. - **The 16:00–16:15 and 17:00 halt:** an HTF bar is aggregated exactly over the Data1 bars MC stamped into it. - **Mixed tiers in a**

session: the BCS `allowMixed` default (false) downgrades the whole session. This already happens in `FlowEngine`. - **Before the tick window:** BarUpDown automatically, through `OrderFlowCapability`.

Journal. `LADDER Data1 tier {old}->{new} at {time}`, at most once per session per change, and `LADDER HTF Data{n} {time} coverage({c}) -> totals-only`.

Unit tests. 1. BCS T1 tick-rule example: the ladder gives 100.00 = bid 6 / ask 2 and 100.25 = bid 0 / ask 5, so Buy = 7, Sell = 6 and delta = +1. 2. BCS T3 Lee-Ready: delta = +4, Source Quote. 3. An HTF merge of 30 bars equals the sum, and the quality is the worst of the parts. 4. A missing minute gives totals-only. 5. A BarUpDown bar has NaN bid/ask and Σtotal = V.

MC trace (A-LAD). One tick-window day, Data1 1m and Data2 30m. Log `HtfLadder(2,0)` POC, Σtotal vs Data2 `Volume`, and the source. Σtotal should agree within 0.5%. This is logged until P2 runs.

Dependencies: `_OrderFlow.FlowEngine` (futures base), S1, `HtfClosed`, `fillFromHistogram`.

VP-4 Profile entropy, KL divergence and Parkinson volatility helpers

Purpose. These are pure statistics on a profile histogram or a bar window.

Used by: - PS-02: skip if composite entropy > 0.93 ("entropy = new 10-line helper"). - RG-02: entropy skip, the "Hosts" row. - A2-29: Miller–Madow H_norm and n_eff at 11:00 as 250-session same-time percentiles. - A2-30: rolling 30-min H_norm every 5m, and its 15-min slope as a 60-session same-time percentile. - A2-31: D_KL on a 1-tick grid. - A2-32: Parkinson σ. - A2-38 needs trade-size entropy, but that stays gated by BCS §E6 and is **not** built.

Classes and files. `_ProfileStats.Function.CS`, region `// <profile-stats>`. It is a static class with no state, so it needs no laziness. Plus `_EntropyTracker`, which is stateful and created per requested metric.

Formulas. - **Input:** dense `vols[]` on 1-tick rows over [low, high]. Zero rows are included. - **Shannon entropy.** p_i = v_i / Σv, H = −Σ_{p_i > 0} p_i ln p_i, and H_norm = H / ln(n). - By default n = the number of dense rows (`entropyRows = DenseRange`, A2: "over 1-tick rows"). The option `TradedRows` uses only rows with v > 0. Q4. - If n ≤ 1 or Σv = 0, return NaN. - **Miller–Madow:** H_MM = H + (m − 1) / (2N), with m = rows with v > 0. - N is configurable: `Bars` (default) = the contributing bar count; `Trades` = the trade count (ladder tier ≤ TickRule only); `Contracts` = Σv, for which the correction is ≈ 0. Q4. - H_norm,MM = min(1, H_MM / ln n). - **n_eff:** 1 / Σ p_i². - **Which variant each idea uses:** RG-02 / PS-02 use plain H_norm (the book threshold 0.93). A2-29 / A2-30 use H_norm,MM. - **KL (A2-31):** D_KL(P‖Q) = Σ p_i ln(p_i / q_i). 1. Use a common 1-tick grid covering both ranges. 2. Add ε = 1e-6 to every row of both, then renormalise. 3. The result is in nats. The order is P = today's developing RTH profile and Q = the prior full RTH profile. - **Parkinson:** σ_P² = (1 / (4 n ln 2)) Σ_i (ln(H_i / L_i))² over bars with L_i > 0. H = L bars count as 0. - `ParkinsonPerBar(bars)` returns σ_P (fractional per bar). - `ParkinsonSessionPrice(bars, refPrice)` = refPrice × σ_P × √n. - A2-32 uses refPrice = the session VWAP and needs ≥ 60 5m bars, otherwise NaN. The session scale is the default for A2-32 (Q5). σ̂ for tomorrow uses the same definition, so the scale choice shifts all z_k uniformly. - **5m bars without a 5m chart:** `AggregateBars(ring, TimeSpan.FromMinutes(5), t0, t1)` builds clock-aligned 5m OHLC from Data1 1m. This is **statistic-only**: no rule trades on it and it is not an HTF stream. Q6 asks whether that is acceptable under MFS §3's no-internal-HTF rule. - **`_EntropyTracker`** (A2-29, A2-30): - It keeps a ring per clock-minute slot (`todMinuteOfSession`) of the last L sessions' values: L = 250 for A2-29 and 60 for A2-30. - `Percentile(slot, x)` uses **strictly earlier** sessions only. It is the mid-rank: 100 × (count(< x) + 0.5 × count(= x)) / count. - It needs ≥ 0.8 L samples, otherwise NaN. - The value is pushed at the slot's decision time, after it has been used. - **A2-30 rolling series:** - Sample on each closed Data1 bar whose end minute ≡ 0 (mod 5) inside RTH. - H(t) = H_norm,MM of `RangeProfile(1, t − 30 min, t)`, using bars stamped in (t − 30 min, t]. - H_slope(t) = H(t) − H(t − 15 min). The tracker stores H_slope.

Config (`protected ProfileStatsConfig ProfileStats`). `{ entropyRows = DenseRange, mmN = Bars, klEpsilon = 1e-6, parkinsonMinBars = 60, ewmaSessions = 10 (λ = 2/11), pctLookbackA229 = 250, pctLookbackA230 = 60, pctMinFill = 0.8 }`.

Edge cases. - An empty or single-row profile gives NaN. - Half days: A2-30 sampling stops at the early close. The 11:00 / 12:00 decision times are unaffected. - Roll: KL against a prior profile with `crossesRoll` (a different price space) gives NaN plus a journal line. For Parkinson, the splice bar is excluded. - Holidays: there are no samples, so the percentile rings skip that day. - Data gaps: rows simply have less volume. Coverage comes from VP-1.

Journal. At decision times only: `PSTAT {tag} Hn({x}) Hmm({x}) neff({x}) pct({x})` and `PSTAT KL {d} today||prior({date})`.

Unit tests. 1. Uniform 4 rows: H_norm = 1 and n_eff = 4. 2. One row with all the volume: H = 0 and n_eff = 1. With n = 1, H_norm = NaN. 3. MM with m = 3 and N = 10 adds 0.1 to H. 4. KL(P‖P) = 0. KL([0.5, 0.5] ‖ [0.9, 0.1]) = 0.5 ln(0.5/0.9) + 0.5 ln 5 = 0.5108 nats (tolerance 1e-4 for ε). 5. Parkinson with two bars whose ln(H/L) = 0.02 and 0.04: $\sigma^2 = 0.002 / (8 \ln 2) = 3.607e\text{-}4$, so $\sigma = 0.018992$. 6. Percentile with ties, and NaN below the minimum fill. 7. The A2-30 window boundary is exclusive at t − 30.

MC trace (A-PS). Log the 11:00 H_norm / n_eff and the 12:00 KL for 20 sessions. The G-V Python recomputes them from the exported 1m bars and must match to 1e-9.

Dependencies: VP-1, `_BarRing`.

VP-5 FVG (fair value gap) detector

Purpose. Detect three-bar imbalance gaps and track their lifecycle.

Used by: - PS-14 (with DV-05 merged): "FVG detector (✗ trivial)". - DV-08: HV zone + FVG; the level = high(bar1) if |high(bar1) − the near HVN edge| ≤ 0.2 × ATR30m. - PS-20 says single prints ≈ FVG. That is an optional overlap report.

Classes and files. `_FvgDetector.Function.CS`, region `// <fvg>`, containing `FvgConfig`, `Fvg` and `_FvgDetector`. The host is `_CustomSignal`, because no VP data is needed.

Definition. Bars 1, 2, 3 are consecutive closed bars on the stream, with bar 3 the newest. - **Bullish:** $Low_3 > High_1$. The zone is [$High_1$, Low_3], and `start` = $High_1$. - **Bearish:** $High_3 < Low_1$. The zone is [$High_3$, Low_1], and `start` = Low_1. - **Size filter:** size = hi − lo, and it must be ≥ max(minTicks × tick, minAtrFrac × ATR). The ATR is the detector's own Wilder ATR(atrLen) on that stream. - An FVG is created at bar 3's close and is usable from the next bar (`createdTime` = $time_3$).

Lifecycle (checked on each later closed bar). - *touched*: the first bar trading into [lo, hi]. - *fillPct*: the deepest penetration / size. - *filled*: with `fillOn = Close`, a close beyond the far edge (for bullish, close < lo). With `Trade`, the low trades below lo. - *expired*: age > `maxAgeSessions`. - *invalid*: a roll.

Config (`protected FvgConfig Fvgs`).

Field	Default
`enabled`	false
`dataStream`	1
`minTicks`	1
`minAtrFrac`	0
`atrLen`	14
`crossSessions`	false: all 3 bars must have the same `tradingDayKey`, and bar 3 cannot be the session's first bar, so an open gap is never an FVG
`rthOnly`	false
`maxLive`	20 (oldest evicted)
`maxAgeSessions`	5
`fillOn`	Close

API. `IList<Fvg> Live`, `Fvg NewThisBar`, `Fvg Nearest(double price, int dir)`, `LevelRef ToLevel(Fvg f, bool startEdge)`. The `Fvg` record holds `{ id, dir, lo, hi, start, createdTime, bar1Time, touched, firstTouchTime, fillPct, filled, expired, invalid }`.

Edge cases. - Roll: invalidate all and journal. - Equal edges (Low_3 == $High_1$) are not a gap. - Time gap: if $time_3 - time_1$ > 2.5 × the bar resolution (missing bars, or the 16:15–16:30 / 17:00 halts), there is no FVG. - On holidays, Globex bars still count unless `rthOnly`.

Journal. `FVG new {U|D} [{lo}-{hi}] size({t}t) at {time}` and `FVG filled|expired|invalid(roll) {id} at {time}`.

Unit tests. 1. Bull and bear zones exact. 2. An equal edge gives none. 3. The size filters. 4. The session boundary and the time-gap exclusion. 5. A fill on Close vs on Trade. 6. Roll invalidation. 7. maxLive eviction.

MC trace (A-FVG). An indicator twin draws the live FVGs for 5 sessions. Check them by eye and log the counts.

Dependencies: `rollOn`, `tradingDayKey`.

VP-6 Other VP helpers the Needs lines require

The catalogue files these under "M-GMM (deferred)" (A2-25, A2-26) or lists them as minor (A2-34). They are histogram helpers, so they live in `_ProfileStats` and use VP-1.

(a) Block-bootstrap POC (A2-25). `BootstrapPoc(IList<RingBar> sessionBars, int block = 5, int draws = 500, ulong seed) → { medianPoc, ci90Ticks, stability }`. - **Moving-block bootstrap:** - Each draw concatenates ⌈n / block⌉ blocks. Each block starts uniformly in [0, n − block], and the result is truncated to n bars. - Rebuild the profile with the VP-1 distribution, and take the POC with `ProfileMath.PocIndex`'s tie rule. - ci90 = P95 − P5 of the draw POCs, in ticks, using linear-interpolation percentiles (the numpy default). - stability = the share of draws with |POC − median| ≤ 1 tick. - Seed = `_Rng.Seed(25, tradingDay.Ticks, block, draws)`. - The cost is about 1.6M row adds per session at the close. - Emits `BootstrapPocZoneLo/Hi` = POC ∓ ci90/2.

(b) KDE bandwidth sweep (A2-26). `KdeSweep(prices, vols)`. - σ = the volume-weighted SD of the row prices. - h_j = σ × (0.05 + j × 1.45 / 29) for j = 0..29. - $f_j(x) = \Sigma_i v_i \varphi((x - x_i) / h_j)$ on the 1-tick grid, with the kernel truncated at 5 h_j. - POC_j = argmax f_j (ties go to the lowest price), $J_j = |POC_j - POC_{j-1}|$, med = median J, and range = max POC − min POC. - **Verdict:** - BIMODAL if max J > 5 med; - STABLE if max J < 2 med **and** range < 0.15 σ; - otherwise FLAT. - If med = 0, use 1 tick (Q7). - **Disagreement flag:** |MC POC − POC at h_{cv}| > 4 ticks. h_{cv} = the argmax over j of the leave-one-row-out weighted log-likelihood.

(c) V/T urgency prices (A2-34). - V(p) = the row volume, and T(p) = the number of 1m bars with L ≤ p ≤ H. - The TPO POC = argmax T. - Output the top 3 V/T rows that are ≥ 0.5 × ATR30 from both the volume POC and the TPO POC, as `VtPrice` levels.

(d) Profile mixtures (A2-27, A2-28, A2-32). These are in G-P (§3.2), and share the EM core with the regime engine.

Laziness. Each runs on call only and is memoised per (tradingDay, config).

Journal. `PSTAT BOOT poc({p}) ci90({t}t) stab({s})`, `PSTAT KDE {verdict} maxJ({..}) medJ({..}) range({..}) cvPoc({..})` and `PSTAT VT {p1,p2,p3}`.

Unit tests. - Bootstrap on a one-row profile gives stability 1 and ci90 0. The same seed gives identical output, and a different seed gives a different one. - KDE on two well-separated row clusters gives BIMODAL. A single Gaussian gives STABLE. - A V/T hand example.

Offline check. The G-V Python ports SplitMix64. Bootstrap must match exactly, and KDE to 1e-9.

---

2. Small items (priority 2)

S1 Add the MC-BidAsk tier to `_OrderFlow.FlowSource`

Purpose. Rev 2.7 VPF-5 inserts tier 2, "MC-BidAsk (High)", so that MC's own Ask/Bid-traded split is preferred over our Lee-Ready, which stays as the cross-check and fallback (BCS rev 2.7 §B).

Used by: - OF-04 and OF-06 (tier ≥ 3); - OF-02 (delta z, W-T); - `_StackedImblCont*`, `_AbsorptionRev*` and `_DeltaDivRev*`; - VP-3.

Change. 1. `enum FlowSource { Aggressor = 0, McBidAsk = 1, Quote = 2, TickRule = 3, BarUpDown = 4, Unavailable = 5 }`. "Lower = better" is kept. - Before committing, run `rg -n "FlowSource" *.CS tests/` and check that there are no ordinal literals or casts. The current code compares by name (`_OrderFlow.Function.CS` l.114, 243–297; `_VolumeProfileFuturesBase` l.503). - The worst-of logic at l.297 must compare by enum order. 2. `HasLadder` also accepts `McBidAsk`. 3. Add `FlowConfig.McBidAsk` (`enum EMcBidAsk { Off = 0, Prefer = 1, CrossCheckOnly = 2 }`), **default Off** until the P-BA probes pass. With `CrossCheckOnly`, the levels are loaded and logged for T9 while Quote stays the source. 4. Add `FlowConfig.mcVolTolerance = 0.005`. 5. `FlowEngine.AddMcLevels(DateTime barTime, double[] price, double[] ask, double[] bid)` queues a level set by bar time. In `CloseBar`, a set whose time equals the bar's `CloseTime` and whose |Σ(ask + bid) − bar volume| / bar volume ≤ tolerance becomes the Ladder, with Source = McBidAsk and Quality = High. Otherwise the bar falls through to Quote / TickRule. 6. **Ingestion (futures base).** When `McBidAsk != Off`, issue one more `DataLoader.BeginLoadData`. The request is a copy of `Bars.Request` with the Volume Delta chart type set in `Resolution.ChartType`. Read `IDataLoaderResult.Data2` (`BarPriceLevels[]` with `Bar`, `Levels`, `Index`, `Time`, `TickID`; PG §4.7.4 p75–76) for history and `RTData2` for real time. - PG fact (§4.7.4): "If either VolumeDelta or CumulativeDelta are specified as ChartType in the request, then the information on Data2 will be relevant … if real-time is requested then RTData2 is required." - **PROBE P-BA1:** the exact `ChartType` member names for Volume Delta. The PG names them only in prose. - **PROBE P-BA2:** the members of `PriceLevel`. The PG shows only `PriceLevel[] Levels`, "VolumeProfile/Volume Delta data". The VP `ILevel` has `AskTradedValue`, `BidTradedValue`, `TotalValue` and `Price` (PG p84–85), but whether `PriceLevel` has the same fields is unverified. Read the metadata as was done for P5. - **PROBE P-BA3:** whether `BarPriceLevels.Time` equals the Data1 bar stamp; the history depth on dxFeed; and the request size and throttling for about 2 years (BCS §E2). - **PROBE P-BA4:** whether the chart or request must be in Ask/Bid-traded mode, as BCS asserts. The PG is silent. 7. **Method tag:** `McBidAsk(MC rule: inside-spread trade → last recorded side, UM p35/p159)`.

Lazy activation. No request is made unless `OrderFlowEnabled && FlowCfg.McBidAsk != Off`.

Edge cases. - Missing levels for a bar: fall through, and count it. - Roll: nothing (per bar). - Before the first MC level date: not available. - Real-time vs history: BCS T5 parity is extended to McBidAsk.

Journal. - `OF McBidAsk loaded bars({n}) first({t}) last({t})`. - Per session: `OF T9 session({d}) signAgree({pct}) bars({n}) mcCover({pct})`. - `OF tier McBidAsk->Quote reason({volMismatch|missing})`, once per session.

Unit tests. 1. The enum order, and the worst-of rule (McBidAsk + Quote → Quote). 2. An `AddMcLevels` override with exact volume. 3. A tolerance miss falls through. 4. All existing `tests/OrderFlowTests` pass after the renumbering.

MC acceptance (T9). On a replay day, MC-BidAsk and Lee-Ready bar delta agree on sign on ≥ 90% of bars. The threshold is BCS's guess, so log the real figure.

Dependencies: the futures-base DataLoader path (`_VolumeProfileFuturesBase` l.680–700).

S2 M-DAILY: HV6 / HV20 / HV100 and RSI(3) of ROC1

Used by: - **A2-53:** HV(6 d) < HV(100 d) with HV = the SD of daily log returns. - **A2-55:** HV6 < HV100. - **A2-70:** HV20 veto with hysteresis, plus the trailing 2-year 90th-percentile arm. - **A2-56:** LBR = Wilder RSI(3) of ROC1. The brief attributes RSI(3)/ROC1 to A2-53, but the catalogue's Needs line for it is **A2-56** (l.2655–2667).

Changes in `_DailyContext.Function.CS` / `DailyConfig`. - New fields: - `int[] hvLengths = {}` (empty = off; e.g. {6, 20, 100}); - `int hvAnnualisation = 252`; - `int hvPctLookback = 0` (0 = off; 504 for A2-70); - `int lbrRsiLength = 0` (0 = off; 3 for A2-56); - `EDailyClose closeBasis = RthValue`. - **Close basis.** A2 defines returns on **RTH close-to-close** (A2 l.926; A2-56: "RTH closes"). - With `dataStream = 0` (aggregate Data1), the day's RTH close = the close of the last bar stamped ≤ `RthValueEndEt(day)` (16:00 ET, or the early close). - On a daily Data n stream, what the daily close is (settlement vs last) is **PROBE P3**, which is still open. In that case journal `DAILY closeBasis=Bar (P3 unverified)`. - **HV(n):** - r_t = ln(rc_t / rc_{t−1}) on the RTH-close series, **ratio-adjusted** for rolls (`Day.rc`, MFS §0.3). - HV_n = √252 × the sample SD (n − 1) of the last n completed days' r. It is NaN until n + 1 closes exist. - Keep one return ring of max(hvLengths, hvPctLookback + 20), compute on query in O(n), and memoise per day. - **HvPercentile(n):** the rank of today's HV_n among the previous `hvPctLookback` daily HV_n values, using mid-rank. - **ROC1 and LBR:** - ROC1_t = C_t − C_{t−1} in points, on the **difference-adjusted** RTH close (the level series), so a splice is not a fake move. - LBR = the Wilder RSI(3) **of the ROC1 series**. With d_t = ROC1_t − ROC1_{t−1}: up_t = max(d, 0) and dn_t = max(−d, 0). - Seed avgUp / avgDn with the SMA of the first 3 values. Then avg ← (2 avg + x) / 3, and RSI = 100 − 100 / (1 + avgUp / avgDn). - When avgDn = 0, the result is 100, or 50 if avgUp = 0 as well (the same conventions as `Rsi2`). - **Q8:** the default reads "RSI(3) of ROC1" as an RSI computed on the ROC1 series. The alternative reading is an RSI(3) of the closes, which is just `rsiLength = 3`. - **API:** `double Hv(int n)`, `double HvPercentile(int n)`, `double Roc1`, `double Lbr`, `double RthClosePrev`. All are NaN if not configured, with a one-time warning.

Edge cases. - Roll: handled by the adjusted series. The splice-day return is kept, but computed on the adjusted close. - Half day: the early-close RTH close is used. - Holiday: no day, and no zero return. - A gap of more than 4 business days: the return spans the gap, and a journal line is written.

Journal. `DAILY {date} hv6({x}) hv20({x}) hv100({x}) hv20pct({x}) lbr({x})`, once per day when enabled.

Unit tests. 1. A constant 1% return gives HV = 0. 2. An alternating ±1% series gives the hand-computed sample SD × √252. 3. A 10-day hand series for LBR, with the expected values in the test comments. 4. A roll offset gives the same HV as the same data with no gap. 5. The percentile. 6. RTH close selection on a half day.

MC trace (A-HV). One year of HV20 and LBR is exported and compared with pandas on the exported RTH closes, to 1e-9.

S3 AVWAP placebo twins P1 (unweighted) and P2 (volume-shuffled)

Purpose. An idea earns the "AVWAP" label only if it beats P1 (the unweighted average from the same anchor), P2 (the volume-shuffled AVWAP) and P3 (EMA20) by ≥ 0.05 R (A2 l.35–36, l.97; catalogue l.148). P3 is a plain EMA and needs no engine work.

Used by: every ✓ M-AVWAP idea (the AV family, including merged A2-01, A2-05 and A2-07), A2-14, A2-15, and A2-79 test 4 ("P2-style" volume shuffle).

Change in `_AvwapManager.Function.CS`. - Add `[Flags] enum EAvwapPlacebo { None = 0, P1 = 1, P2 = 2 }` and `AvwapLimits.placebos = None`. - Per anchor, keep twin accumulators. TP is the same as `_VpVwapEngine`: TP = (H + L + C) / 3. - **P1** = ΣTP / n. Its σ_P1 = √(ΣTP²/n − P1²). - **P2** = ΣTP·ṽ / Σṽ. Its σ uses ṽ weights. - ṽ_t = the volume of the bar in the same clock-minute slot (`todMinuteOfSession`) from a **different, earlier** session. The pool is the last `TodSessions` (20) sessions' volumes for that slot, excluding the current session. - The draw is `ṽ = pool[rng.NextInt(pool.Count)]`, with the rng seeded `_Rng.Seed(anchorId, barTime.Ticks, 2)`. This is deterministic and has no lookahead. - **Lazy dependency:** P2 needs a per-slot volume pool. - Expose a read-only `IList<double> VolumePool(int slot)` from `_TodNormaliser` Volume (M-TOD keeps the same-slot history for its z-scores). Enabling P2 enables M-TOD Volume transitively (MFS §0.1 rule 3). - If the pool has fewer than 10 values, use ṽ = the bar's own volume and set `p2Degraded` (journaled once per anchor). - **Swing back-fill:** the existing PV back-fill ring also carries TP and the slot, so the twins can be back-filled identically. - **API on `_Avwap`:** `double P1, P2, P1Sigma, P2Sigma; bool p2Degraded`. A strategy runs a placebo arm by reading `P1` or `P2` in place of `Value`, through its own Variant switch.

Edge cases. - A roll invalidates the twins with the anchor. - An anchor spanning sessions is fine, because the pool is per slot. - After an early close, those slots simply never occur. - A V = 0 bar still counts in P1 and contributes ṽ to P2.

Journal. The create line gets `placebos(P1,P2)`. The trade tag becomes `AV:{type}:{yyyyMMddHHmm}:P1|P2` when a placebo arm is traded.

Unit tests. 1. P1 equals the mean TP. 2. A pool of all-equal values gives P2 = P1. 3. P2 is reproducible for a fixed seed. 4. The pool never contains the current session (asserted). 5. A roll clears the twins. 6. The back-filled swing anchor equals a forward-built one.

MC trace (A-AVP). One anchor's Value, P1 and P2 for 50 bars, run twice, with identical P2 values.

Dependencies: `_TodNormaliser` (pool accessor), `_Rng`.

S4 AVWAP roll invalidation available beyond the futures base

Problem. `_CustomSignal.rollOn(ds)` returns false (l.995), and only `_VolumeProfileFuturesBase.rollOn` (l.574) detects rolls, and only for Data1. So on `_CustomSignal` / `_VolumeProfileBase` strategies, and on any Data n stream, `_AvwapManager.OnRoll` and `_PriceActionEngine.OnRoll` never fire. This is the §2.4a gap.

Change. 1. Extract the VPF-4 detector into `_RollDetector.Function.CS`, region `// <roll-detector>`. The logic is unchanged: - **Series:** a new trading-day open that gaps ≥ `RollGapTicks` × tick from the prior close on days 1–21 of Mar/Jun/Sep/Dec, **or** a change in `Bars.Info.Name`. - **Table:** `RollDates.Contains(tradingDay)`. - Its inputs are `(barNumber, tradingDay, open, prevClose, name, tick)`. 2. Move `RollDetection` (default Off), `RollGapTicks` (8), `RollDates` and `RollBars` into `_CustomSignal`. The trading day comes from `tradingDayKey`, which `_VolumeProfileFuturesBase` already overrides with `GlobexDate`. The futures base keeps its overrides by delegation, and `profileCrossesRoll` stays in place. 3. `_CustomSignal.rollOn(ds)`: - For ds = 1, run the detector (idempotent per bar). - For ds > 1 on the **same root symbol** (the MFS §3.6 check), set a pending flag at each Data1 roll and report true on that stream's first closed bar with Time ≥ the Data1 roll bar time. - A cross-market leg with a different root does not inherit it. That leg needs its own detector, which is out of scope here. 4. Warning: if `Bars.Info.Category` says the symbol is a future and `RollDetection == Off`, journal one Warning. - **PROBE P-R1:** the enum value that identifies futures (the PG lists `Info` members but does not confirm the value names). If the probe is unresolved, skip the warning.

Unit tests (`tests/RollTests`). 1. The ported detector gives identical roll bars on a synthetic series, compared with the current futures base output. 2. Table mode. 3. A name change. 4. The Data2 pending flag fires on the first Data2 close at or after the Data1 roll. 5. Off gives no rolls.

MC trace (A-ROLL). A `_CustomSignal`-derived strategy on `@ES` 1m with Data2 60m, over the roll days in the RollDates table for 2025–26. The journal shows `AV invalidated (roll)` and the PA OnRoll lines on both streams.

S5 A second PriceAction engine instance on another timeframe

Used by: - MT-01 (60m → 15m → 5m HL at value) and SP-06: "◐ M-SWING (multi-TF: hosted instance + a strategy-owned `_PriceActionEngine` on Data n)". - MT-02, plus HTF-swing AVWAP anchors.

This item makes the Data n instance **hosted**, so roll, laziness and journal handling are uniform.

Change in `_CustomSignal`. - Add `protected List<PriceActionConfig> ExtraPriceAction { get; } = new List<PriceActionConfig>();` and a new field `PriceActionConfig.role` (default ""). Each entry sets `enabled = true` and `dataStream = n`. - **StartCalc:** for each entry, `requireDataStream`, then create, `Configure()` and throw on error. This is the l.1016 / l.634 pattern, with the label `"PA{ds}-{role}"`. Apply the futures preset (MFS §1.8) on the futures bases, unless the entry overrides it. - **Sharing:** an instance is shared between consumers with the same `(dataStream, config.Canon())`. The consumers are the main PA, the AVWAP's PA (l.1035) and the XMKT leader swing (l.634). - **CalcBar:** after the main PA, call `if (rollOn(ds)) pa.OnRoll(time); pa.Step(BarsOfData(ds));` for

each engine. The engines already de-duplicate by CurrentBar and closed status. - **Accessor:** `internal _PriceActionEngine PriceActionFor(int dataStream, string role = null)` returns null with a one-time warning if missing. The engine state is as of its last closed bar, which gives the `HtfClosed` semantics.

Edge cases. - Roll: per S4. - Missing HTF bars on holidays: no step. - Same-timestamp closes of Data1 and Data n: the ordering is Probe P2. Rules must read HTF state only through closed bars, per MFS §0.4.

Journal. `PA{ds}-{role} created strength({s}) layers({l}) shared({0|1})`, plus the existing labelled confirmation and roll lines.

Unit tests (PriceActionTests). 1. Two instances on different streams are independent. 2. Identical configs share one instance (reference-equal). 3. OnRoll fires on Data2 through the S4 flag.

MC trace (A-PA2). Data1 5m and Data2 60m. The 60m pivot confirmations from `PriceActionFor(2)` equal the `_PriceAction` indicator on a 60m chart.

---

3. M-GMM (priority 3)

3.0 Scope: what the catalogue's "M-GMM" covers (Q2)

The catalogue tags **eight** ideas "✗ M-GMM (deferred)": **A2-25, A2-26, A2-27, A2-28, A2-29, A2-30, A2-31 and A2-32** (l.2118–2269). Their Needs lines are all *profile statistics*: bootstrap, KDE, a volume-weighted GMM with BIC, entropy and KL. None of them needs a daily market-regime model. The brief asks for a market-regime engine, so this section specifies **one shared EM core with two consumers**:

- **G-P (profile mixtures):** a weighted 1-D GMM on profile rows, for **A2-27, A2-28 and A2-32**. The other five (A2-25, A2-26, A2-29, A2-30 and A2-31) are fully served by VP-4 and VP-6 and need no mixture model.
- **G-R (market regime engine):** a daily Gaussian HMM (or GMM) on returns, volatility and trend. It publishes probabilities, a label and `pm.regime`.
- Catalogue consumers: RG-03 (a volatility-regime stratifier; G-R is the model-based alternative, and the ATR tercile stays as the control and the fallback), PF-02 (regime-complementary sleeves), A2-70's market-adaptive arm, A2-40's regime-bucketed p90 stop slippage, A2-28's trend-state disable (as a daily prior only), and M-PORT `pm.regime` (m-port-spec l.72).
- **No catalogue idea currently requires G-R.** It is infrastructure, and it must be pre-registered as a stratifier before any idea gates on it.

3.1 Shared EM core `_Gmm` (pure C#)

- **File:** `_Gmm.Function.CS`, region `// <gmm>`. It is pure managed C# with no external library.
- The PG does allow external assemblies, provided they are added through PLEditor (PG p88: "For any study it is possible to add a reference to any other assembly … Any reference to the other assemblies should be added ONLY via PLEditor .NET"), and M-PORT already uses Newtonsoft.
- No library is needed here. If one is ever proposed (e.g. MathNet), **PROBE P-G1** must check that it also loads under Portfolio Trader.
- **Types:**
- `GmmModel { int K, D; double[] w; double[][] mu; double[][,] cov; double logLik, bic; int iters; bool converged; int reseeds; };`
- `GmmFitConfig { kMin = 1, kMax = 3, restarts = 10, maxIter = 200, tol = 1e-6, tolMode = Relative|PerSample, regCovar = 1e-6, minWeight = 1e-3, bicDelta = 10, ulong seed }`.
- **Weighted EM** (weights $\omega_n \geq 0$, scaled so $\Sigma\omega = N_{eff}$; for G-R, $\omega = 1$):
- **E step:** $\ln r_{nk} = \ln w_k + \ln \mathcal{N}(x_n \mid \mu_k, \Sigma_k) - \text{logsumexp}_j(\cdot)$.
- **M step:** $N_k = \Sigma\ \omega_n r_{nk}$, $w_k = N_k / \Sigma\omega$, $\mu_k = \Sigma\ \omega_n r_{nk} x_n / N_k$, and $\Sigma_k = \Sigma\ \omega_n r_{nk} (x_n - \mu_k)(x_n - \mu_k)^T / N_k + \text{regCovar}\cdot I$.

- **Stop** when |ΔlnL| / |lnL| < tol (Relative), or when the change in mean lnL per sample < tol (PerSample, the sklearn convention, used for G-V).
- **Cholesky** is hand-coded for D ≤ 4. If it fails, retry with regCovar × 10, up to 3 times. After that the component is degenerate and is re-seeded.
- **Init:** k-means++ seeded from `_Rng` (seed + restart index), then 10 Lloyd iterations. The covariance = the within-cluster covariance + reg, and w = the cluster shares. For G-V, our init is **passed to the reference** (`weights_init`, `means_init`, `precisions_init`), so the k-means does not need to match sklearn's.
- **Collapse:** if N_k / Σω < minWeight, re-seed μ_k at the point with the lowest current likelihood, with the global covariance. A re-seed in the final iteration means `converged = false`.
- **BIC:** p = (K − 1) + K·D + K·D(D + 1)/2 and BIC = −2 lnL + p ln N_eff.
- Choose K by starting at kMin and moving to a larger K only if it lowers BIC by more than `bicDelta` (A2: "prefer the smaller K unless BIC improves by > 10").
- Keep the best lnL across restarts for each K.
- **Ordering (label stability):** after the fit, sort the components by μ of the ordering feature (G-P: price ascending; G-R: f2 log-vol ascending, ties broken by f3), and remap all parameters.

3.2 G-P Profile mixture (A2-27, A2-28, A2-32)

- **API (in `_ProfileStats`, using `_Gmm`):** `ProfileMixture(double[] prices, double[] vols, int kMax, ulong seed)` → `{ GmmModel (D = 1), mu[], sd[], pi[], voidPrice[], voidDepth[] }`.
- **Data:** x_n = the row price and ω_n ∝ the row volume, from the VP-1 RTH profile (uniform distribution).
- **N_eff (Q9):** Σω is scaled to the **number of contributing 1m bars**. Raw contracts would make the ln N penalty meaningless. The scaling is configurable: `Bars | Rows | Contracts`.
- **K:** 1..3 for A2-27/28 and ≤ 4 for A2-32. 10 restarts, ΔBIC > 10.
- **Void:** between adjacent μ_k and μ_{k+1}, void = the argmin of the mixture density f on the tick grid. depth = 1 − f(void) / min(f(μ_k), f(μ_{k+1})). A2-27 needs depth ≥ 0.5.
- **Edges:** μ ± c·σ, with c ∈ {0.5, 1.0} as the A2-27 parameter.
- **A2-32 construction:**
- Take the last 20 sessions inside one contract. If a roll falls in the window, return NaN and journal "insufficient window" (A2-39).
- For each session, map every 1m bar's uniformly distributed volume to z = (p − sVWAP_s) / σ_s, where σ_s = `ParkinsonSessionPrice` on that session's 5m bars (≥ 60 obs).
- Bin at Δz = 0.05 and pool the bins.
- Fit with K ≤ 4.
- Tomorrow's levels: level_k = the prior sVWAP + z_k × σ̂, with σ̂ = the EWMA (10 sessions) of σ_s.
- **Output:** `LevelRef` kinds `GmmMu`, `GmmEdge`, `GmmVoid` and `SigmaMu`, created at the session close and usable from the next session.
- **Laziness:** runs once per session close, only when called.
- **Seed:** `_Rng.Seed(27, tradingDay.Ticks)`.
- **Journal:** `GMMP {date} K({k}) bic({b}) mu({..}) sd({..}) pi({..}) void({p}/{depth})`.

3.3 G-R Market regime engine

Classes and files. `_RegimeEngine.Function.CS`, region `// <regime>`, containing `RegimeConfig`, `_RegimeEngine` and `RegimeState`. The host is `_CustomSignal`. It enables M-DAILY lazily (MFS §0.1 rule 3) with S2's HV lengths {20, 63}.

Features. These are daily values computed at the RTH value close (16:00 ET, or the early close), from M-DAILY's roll-adjusted RTH-close series (S2).

#	Feature	Formula	Role
f1	Return	r_t = ln(rc_t / rc_{t−1})	direction and shock
f2	Log realised vol	ln(HV20_t) (S2)	volatility level (RG-03, A2-70)
f3	Trend	ln(rc_t / rc_{t−63}) / (HV63_t × √(63/252))	trend vs chop (scaled TSMOM63)
f4 (opt., off)	Range vol	ln(Parkinson σ of the last 10 daily bars)	robustness arm

- **Standardisation:** z-score each feature with the mean and SD of the **current fit window**. These are frozen until the next refit.
- **Validity:** an observation needs every feature finite, which takes ≥ 64 adjusted closes.

Model. - **Default** `method = Hmm`: a Gaussian HMM with full covariance and **K = 3** (calm / normal / stressed). - `method = Gmm` treats days as i.i.d. with the same EM. - `kSelect = Fixed` (default 3). The option `Bic` (K = 2..4) is for research only, because K changing across refits breaks label continuity. - **HMM fit (Baum–Welch):** - Emissions use the §3.1 M step with the forward–backward posteriors γ over **the fit window only**. - Transitions: A_ij = (Σ ξ_t(i, j) + transPrior) / (Σ_t γ_t(i) + K·transPrior), with transPrior = 1. - $\pi_0 = \gamma_1$. - The forward/backward passes use Rabiner scaling, and lnL = Σ ln c_t. - Init: the GMM fit (10 restarts), then A = 0.9 on the diagonal and 0.1 / (K − 1) elsewhere. - Iterate up to 200 times, tol = 1e-6. - Smoothed γ is used **only to estimate the parameters inside the fit window**. It is never published.

Walk-forward schedule (no lookahead). - **Fit window:** the trailing `fitSessions = 1000` valid days ending at the refit day's close. The window expands from `minSessions = 500` until 1000 are available. - **Refit:** `refit = Monthly`, run after the close of the first trading day of each month. The options are `EveryNSessions` (21) and `Quarterly`. The new parameters apply from the **next** observation on. - **Published value for day t:** the **filtered** posterior P(s_t | x_1..x_t) under the parameters of the latest refit made at a close ≤ t − 1. It comes from the forward recursion, $O(K^2)$ per day. For GMM, it is the per-day posterior. - **After a refit:** re-initialise the filter by running the forward pass over the last `warmFilter = 63` observations with the new parameters, which avoids a cold start. - **Truncation invariance (test G-T1):** the outputs for day t are bit-identical whether the data ends at t or at any T > t.

Label stability. 1. **Ordering:** states are ordered by the f2 mean ascending, so 0 = the lowest vol and K − 1 = the highest. Ties are broken by the f3 mean. 2. **Cross-refit check:** match the new states to the old by the minimum total Bhattacharyya distance (brute force over K! ≤ 24 permutations). If that matching disagrees with the vol ordering, the ordering still decides. The engine journals `REGIME relabel-conflict` and counts it as a model-health metric. 3. **Hysteresis on the published label:** switch from L to j only if p_j ≥ `switchProb` (0.60) and p_j − p_L ≥ `switchMargin` (0.10). `RawArgmax` is exposed without hysteresis.

Outputs (`RegimeState`, copy on read). - `bool Valid; int Label` (0..K−1, or −1 when invalid); `int RawArgmax; double[] Probs; DateTime AsOf`. - `DateTime FittedThrough; int K; double LogLik; int Refits; bool UsingFallback`. - **Timing:** the value is computed at day t's close and is **usable from the next bar** (the next session). A rule must never use day t's value to trade day t's RTH. - **Accessor:** `protected RegimeState Regime` returns a neutral value plus a one-time warning when disabled. - **`pm.regime` (M-PORT):** this is the `protected virtual int PortfolioRegime` in m-port-spec l.72 (default 0 = unknown). - With `RegimeConfig.enabled && publish`, the base default returns `Label + 1` (1..K), so 0 still means unknown. It is written at the session close as the spec says. - The probabilities go to an *optional* key `pm.regimeP` (`double[]`, copy-on-write). That key is **not in m-port-spec** (Q10). - The MM treats a stale `pm.stamp` as unknown. - M-PORT's `publishPortfolio()` is not yet coded (§2.4a), so G-R publishing lands with it or after it.

Config (`protected RegimeConfig RegimeConfig`).

Field	Default
`enabled`	false
`method`	Hmm
K	3
`features`	{Return, LogHv20, Tsmom63z}
`fitSessions`	1000
`minSessions`	500
`refit`	Monthly
`restarts`	10
`maxIter`	200
`tol`	1e-6
`regCovar`	1e-6

Field	Default
`transPrior`	1
`warmFilter`	63
`switchProb`	0.60
`switchMargin`	0.10
`seed`	20261021
`fallback`	AtrTercile
`publish`	false

Fallback. It applies with fewer than minSessions valid days, or when every restart fails or does not converge. - `fallback = AtrTercile`: the RG-03 rule. The ATR20d percentile over 250 days gives a tercile, mapped to labels 0/1/2 (for K = 3). Probs are one-hot and `UsingFallback = true`. - `fallback = Unknown`: Valid = false, Label = −1, and pm.regime = 0. - A failed *re*-fit keeps the previous parameters and journals.

Data guidance. - With aggregate Data1 1m from ~2016 (dxFeed minute history, BCS rev 2.7 §B), the first valid regime comes after about 500 sessions (≈ 2 years), and the full 1000-session window after about 4 years. - W-D research (2000+) needs a daily Data n stream, so P3 must be resolved first.

Edge cases. - **Roll:** the features use roll-adjusted series, so the splice day is a normal observation. A sanity check journals `roll adjust suspect` when |z(f1)| > 5 on a roll day. - **Holidays:** no observation. - **Half days:** a normal observation at the early close. - **A gap of more than 4 business days:** the observation is kept as one HMM step and journaled (Q11). - **StopCalc / recalc:** rebuilt from scratch, and determinism makes it identical.

Laziness and cost. Nothing runs unless enabled. The per-day cost is O(K²) for the filter plus the features. A refit (about 1000 obs × 200 iterations × K × D³, × 10 restarts) is well under a second and runs at the close.

Journal. - `REGIME refit {date} window({n}) K({k}) ll({x}) iters({i}) conv({0|1}) means({..})`. - `REGIME {date} label({L}) raw({j}) p({p0,p1,p2}) fb({0|1})`, once per day. - `REGIME switch {old}->{new} {date}`. - `REGIME fallback reason({insufficient|nonconv|chol})`. - `REGIME relabel-conflict {date}`.

Unit tests (`tests/RegimeTests`). 1. **GMM recovery:** 3 known 2-D Gaussians, 3000 samples (`_Rng` + Box–Muller). Means within 0.05, weights within 0.02, with ordering as specified. 2. **HMM recovery:** a simulated 3-state HMM (diagonal 0.95), T = 3000. The fitted diagonal is within 0.03, and the filtered-argmax accuracy is ≥ 85%. 3. **Determinism:** the same seed gives bit-identical output. On separated data, a different seed gives the same ordered labels. 4. **G-T1 truncation invariance:** days 600–700 computed on data ending at 700 vs 900 are identical. 5. **Refit timing:** month starts only, and the parameters apply from the next day. 6. **Fallback:** 300 days gives AtrTercile, equal to an independent RG-03 computation. 7. **Hysteresis:** the label sequence on a hand probability path. 8. **Degenerate data:** duplicate points give a finite fit (reg path). 9. **BIC:** single-Gaussian data gives K = 1. 10. **G-P:** two separated price clusters give K = 2 and void depth > 0.5. A 0.75 / 0.25 split identifies the minor component for A2-28.

G-V Validation against an offline reference. - **Script:** `tests/RegimeTests/reference.py`. It needs numpy, scikit-learn, hmmlearn and pandas, which are not installed on the box (`pip install numpy pandas scikit-learn hmmlearn`). - **Input:** a CSV exported by the C# test or the MC journal (`date, f1, f2, f3, refit`), plus the C# init parameters for each refit. - **GMM check:** `sklearn.mixture.GaussianMixture(n_components=K, covariance_type='full', reg_covar=1e-6, tol=1e-6, max_iter=200, weights_init=…, means_init=…, precisions_init=…)`, with C# `tolMode = PerSample`. - Pass criteria: |ΔlnL| / |lnL| ≤ 1e-6; the ordered parameters within 1e-4; the posteriors within 1e-5. - **HMM check:** `hmmlearn.hmm.GaussianHMM(n_components=K, covariance_type='full', n_iter=200, tol=…, init_params='', params='stmc')`, with our startprob_, transmat_, means_ and covars_. - The transition prior must be aligned to our pseudo-count of 1 per cell (hmmlearn's `transmat_prior` is a Dirichlet α). Verify the convention in the installed version, and document the value used in the script. - Compare `score`, `transmat_` and `means_` after the fit. - hmmlearn's `predict_proba` is **smoothed**. Compare it with our in-window γ. Compare our published **filtered** probabilities with a numpy forward pass on the hmmlearn parameters. - **Walk-forward replay:** a numpy loop reproduces the refit schedule. On ES 2016–2026, ≥ 99.9% of days must have the same label. Differences are allowed only where the two top probabilities are within 1e-6. - **Profile helpers:** the same script ports SplitMix64 and checks VP-4 / VP-6 (bootstrap exact, KDE / entropy / KL to 1e-9).

MC acceptance trace (A-REG). - ES 1m from 2016, with `RegimeConfig.enabled = true` and `publish = true`. - The journal shows a refit on each month start and one REGIME line per day. - In Portfolio Trader, a probe MM signal (on the `_PortfolioRiskManager` skeleton) logs `pm.regime` per strategy per day, and it must equal the chart run. - Export the journal and run G-V.

Dependencies: `_DailyContext` + S2, `_Gmm`, `_Rng`, and M-PORT `publishPortfolio()`.

---

4. Build plan (by priority)

Effort is in Coder-days. Each row merges with green off-MC tests. The MC traces are collected per batch in one probe sitting.

#	Item	Depends on	Effort	Unblocks
1	`_BarRing`, `_Rng`, `_LevelRef` (§0.2–0.6); `ProfileMath.AddRow` made public	—	0.5	all below
2	**VP-1** range profile + handles	1	1.5	PS-13, PS-14, A2-15, PS-12 variant
3	**VP-4** entropy / MM / n_eff / KL / Parkinson + `_EntropyTracker`	2	1	PS-02, RG-02, A2-29, A2-30, A2-31
4	**VP-5** FVG detector	1	0.5	DV-08, PS-14
5	**VP-2** weekly profile stream	base session list	1	MT-02, SW-01
6	**VP-6** bootstrap POC, KDE sweep, V/T	2	1	A2-25, A2-26, A2-34
7	**VP-3** per-bar ladder + HTF aggregation	OrderFlow	1.5	PS-12, OF-04/06 (W-T)
8	**S4** roll detector into `_CustomSignal` + the Data n pending flag	—	0.5	S5, AVWAP on any base
9	**S2** HV6/20/100, HV percentile, RSI(3) of ROC1, RTH close basis	—	0.5	A2-53, A2-55, A2-56, A2-70, G-R
10	**S5** hosted extra PriceAction instances	8	0.5	MT-01, SP-06
11	**S3** AVWAP P1/P2 twins (+ the `_TodNormaliser` pool accessor)	1	1	every AV placebo test
12	**S1** McBidAsk tier (code + CrossCheckOnly); `Prefer` after P-BA1–4	7	1 + probe	OF ideas at tier 2
13	**`_Gmm`** core + **G-P** profile mixtures	2, 3	1.5	A2-27, A2-28, A2-32
14	**G-R** regime engine + `pm.regime` + G-V script	9, 13, M-PORT publish	2.5	RG-03 alt, PF-02, A2-70 arm, A2-40 buckets

Probe sitting (after rows 7 and 12): P-BA1–4, P2 (HTF end stamps; this gates the VP-3 HTF ladder and S5 readings), P-R1, and P3 (only if a daily Data n stream is used). P1 (lazy FunctionSeries) is still open, but nothing here depends on it.

Catalogue updates after merge (for the catalogue owner): - The §2.4a "VP helpers" row becomes Available once rows 1–7 merge. - The M-GMM row is split into "profile statistics (VP-4, VP-6, G-P)" and "regime engine (G-R)". - A2-56's Needs line gets "RSI(3)/ROC1 ✓". - The M-AVWAP row loses the "P1/P2 not built" and "roll only with the futures base" gaps.

---

5. Open questions

1. **Q1:** Does the A2-33 harness already define a level record? If so, `LevelRef` adopts it.
2. **Q2:** The catalogue's eight M-GMM ideas (A2-25…A2-32) are profile statistics, not a market regime. Should G-R be built now as infrastructure (stratifier / `pm.regime`) with no idea gating on it until it is pre-registered, or should it wait behind G-P?

3. **Q3:** For the weekly and multi-week composites, Reset is the default until A2-39 rules. Should the A2-39 cash-terms and back-adjusted arms be built into VP-2?
4. **Q4:** For entropy, is n the dense row count (the default) or only traded rows? Which N goes into Miller–Madow: bars (the default), trades or contracts?
5. **Q5:** For A2-32's σ_session, is it session-scale Parkinson (σ_bar × √n × sVWAP, the default) or per 5m bar?
6. **Q6:** Is the statistic-only 5m aggregation from Data1 acceptable, or must a Data n 5m stream be added? MFS §3 forbids internal HTF bars, but for trading streams.
7. **Q7:** For the KDE verdict when the median jump is 0, is a 1-tick floor acceptable?
8. **Q8:** For A2-56, is the RSI(3) of ROC1 computed on the ROC1 series (the default) or on the closes?
9. **Q9:** For the profile-GMM BIC, what N_eff applies under volume weighting (1m bars by default)?
10. **Q10:** Should `pm.regimeP` (the probability vector) be added to m-port-spec?
11. **Q11:** Should a multi-day data gap be one HMM step (the default) or A^k?
12. **Q12:** Confirm that the G-R settings (K = 3, the features f1–f3, monthly refit, a 1000/500-session window) are fixed now and never swept (M13/M14).
13. **Q13:** The brief attributes RSI(3) of ROC1 to A2-53, but the catalogue says A2-56. Confirm which is meant; S2 serves both.